Train API

Updated - 10-03-2026

Contents

1 Resource Links 2

2 Data Collection / Manipulation 2

3 Architecture 2

4 Authentication 3

5 Generative AI Declaration 3

6 Generative AI Examples 5

1 Resource Links

- GitHub Repository
- Documentation
- Presentation

2 Data Collection / Manipulation

Due to my choice of data set I found that AI would struggle with understanding and parsing it. This is likely because the data is provided by real systems that have complicated documentation that is not included in the models training set giving it no reference points of how to understand the formats presented. The AI is dumb.

Our data was collected from a number of data sources provided by NRE (National Rail) and NWR (Network Rail) through the RDM (Rail Data Marketplace). The following resources were heavily used.

- Network Rail Corpus
- Network Rail Schedule
- National Rail Knowledge Base Stations
- National Rail Darwin Push Port (Archive Provided By David Wheatley)
- National Rail Darwin Timetable

Once our data was collected in its respective formats we could use python scripts provided in the **collection** folder to generate the SQL database needed for our API.

The first operation we must undertake is the creation of the **stations** table. We can run the Python module with the following command.

python -m collection.stations.generate

This command will generate the stations table combining data from Network Rail Corpus and the National Rail Knowledge Base Stations.

3 Architecture

For my API I chose to use the Go programming language. This choice was made on the basis that Go is a compiled static typed language which allows it to be highly performant when compared to APIs defined in other interpreted programming languages such as Python or JavaScript.

To make development of my API in Go simpler I chose to employ an API framework. In my case I chose to use a common well documented framework called Gin. This choice meant there was plenty of documentation of how to use the framework and that a LLM would likely have a good amount of training data related to the framework to infer from.

For my database I chose to use PostgreSQL, this choice was made after trial testing with SQLite. SQLite started to show extreme slowdown during the generation of the dataset, mainly the **stops** table, this forced me to reconsider my choice of DBMS. Instead of operating as a single portable file which

is programmatically controlled through a driver PostgreSQL provides a full fledged engine making it much more performant and able to handle large volumes of traffic. Along with this PostgreSQL allows for more fine grained control of data types such as the use of **VARCHAR**, reducing the overall storage usage of the database.

4 Authentication

Endpoints in the API are organised into three different levels of authorisation.

- Administration - Covers endpoints used by administrators to perform actions such as updating an application approval status.
- Protected - Covers endpoints that the customer can use to manage their account and applications, i.e. creating new applications, refreshing application keys.
- Product - This group covers endpoints that relate to the actual end user data API provided, authentication on these endpoints does not require login and instead makes use of a Bearer token scheme.

Product endpoints rely on the Bearer token authorisation scheme, this means that an application key can be used by an autonomous user to make requests. This allows for the development of external applications which make use of our API. The Authorization header provided when making requests should follow the format **Bearer <base64("<applicationId>:<applicationKey>")>** (Swagger, 2024), an adaptation of OAuth 2.0 Bearer authorisation (Jones and Hardt, 2012).

Administration endpoints make use of login endpoint which returns a JWT (JSON Web Token) (M. B. Jones and Sakimura, 2015) which can be used for authorisation. The token is used on subsequent requests to the administration endpoints. Along with verifying the JWT the service also checks that the JWT claims **"type"** attribute is set to **administrator**, this allows the service to check that the given requester should have access to the administrative endpoints. The claims in the JWT are signed using the services private key so they can be verified as untampered.

Protected endpoints make use of the same JWT (JSON Web Token) authorisation scheme but do not check the **"type"** claim. These endpoints instead verify the validity of the token then check that the customer requesting data has authorisation to access the customer data they are requesting. This is usually that the requested data is the customers own, so the identifier in the claims matches the identifier in the URL.

To sign JWTs (JSON Web Tokens) the service uses a public private Ed25519 key pair and the EdDSA digital signature scheme. This choice was made as Ed25519 presents one of the most secure signature schemes currently adopted in wide use while still being performant for low tier hardware.

5 Generative AI Declaration

Generative AI was used to aid with the development of both the API application itself and the data manipulation scripts. The following Generative AI tools were used.

- T3 Chat (t3.chat)

The T3 Chat tool allows usage of a wide range of LLM models in a chat interface which aided with the development of the application through prompts phrased as questions.

The primary model used through this tool was Claude Sonnet 4.6 due to its high LiveBench (livebench.ai) score and balanced token cost.

"A computer can never be held accountable, therefore a computer must never make a management decision." - (IBM, 1979)

References

IBM. Ibm training manual, March 1979. URL **https://www.ibm.com/think/insights/ai-decision-making-where-do-businesses-draw-the-line**. Accessed: 2026-03-10.

M. B. Jones and D. Hardt. The oauth 2.0 authorization framework: Bearer token usage. RFC 6750, October 2012. URL **https://datatracker.ietf.org/doc/html/rfc6750**. Accessed: 2026-03-10.

J. Bradley M. B. Jones and N. Sakimura. Json web token (jwt). RFC 7519, May 2015. URL **https://datatracker.ietf.org/doc/html/rfc7519**. Accessed: 2026-03-10.

Swagger. Bearer authentication, November 2024. URL **https://swagger.io/docs/specification/v3_0/authentication/bearer-authentication/**. Accessed: 2026-03-10.

6 Generative AI Examples

In python I want to loop over a file and only load what is needed into memory each time, I want to iterate over my file reading up to the next delimiter `<?xml version="1.0" encoding="utf-8"?>` once I reach this I take the previous text up until this point and do some manipulation on it then move onto the next `<?xml version="1.0" encoding="utf-8"?>` delimiter in the file.

To process a large file without loading it all into memory, you can create a **generator function**. This function will read the file line by line (or in small chunks) and `yield` the accumulated text whenever it encounters your specific XML declaration delimiter.

Here is an efficient way to implement this in Python:

```python
import os

def xml_chunk_reader(file_path, delimiter='<?xml version="1.0" encoding="utf-8"?>'):
    """
    Reads a file line by line and yields chunks separated by the delimiter.
    """
    with open(file_path, "r", encoding="utf-8") as f:
        buffer = []
        for line in f:
            # Check if the line contains the delimiter
            if delimiter in line and buffer:
                # If delimiter is found and we have content,
                # split the line to handle text before the delimiter
                before, after = line.split(delimiter, 1)
                if before:
                    buffer.append(before)

                # Yield the accumulated chunk
                yield "".join(buffer)

                # Reset buffer with the start of the next chunk
                buffer = [delimiter + after]
            else:
                buffer.append(line)

        # Yield the final chunk remaining in the buffer
        if buffer:
            yield "".join(buffer)

def process_file(input_path):
    for chunk in xml_chunk_reader(input_path):
        if not chunk.strip():
            continue

        # Perform your manipulation here
        print(f"Processing chunk of size: {len(chunk)} characters")
        # Example: manipulated_data = my_xml_logic(chunk)

if __name__ == "__main__":
    # Path to your large file
    path_to_file = "large_data.xml"
    if os.path.exists(path_to_file):
        process_file(path_to_file)
```

Key Features of this Approach:

1. **Memory Efficiency**: By using `for line in f`, Python uses an internal buffer to read the file, ensuring that even multi-gigabyte files don't crash your RAM.
2. **Generator Pattern**: The `yield` keyword allows you to process one chunk at a time. The program "pauses" its state, performs your manipulation, and then resumes reading exactly where it left off.
3. **Delimiter Handling**: The logic inside the loop handles cases where the delimiter might appear in the middle of a line, ensuring that the `<?xml...?>` header stays with the new chunk rather than the end of the old one.

Mathematical Note on Complexity

If n is the total number of characters in the file and m is the average size of a chunk, the space complexity of this approach is $O(m)$ rather than $O(n)$. This makes it ideal for files where:

$$m \ll n$$